Encontro 5: Comunicação Entre Componentes

Sumário

Encontro 5: Comunicação Entre Componentes 1
1. Introdução 2
   1.1. Por que criar componentes diferentes? 2
   1.2. Quando um único componente pode bastar? 2
   1.3. Riscos do “componente único” (monolito) 2
   1.4. Riscos de “supercomponentizar” 2
   1.5. Regras práticas (rápidas) 3
2. O Fluxo de Dados Unidirecional 4
3. Passando Dados com Props (Pai → Filho) 4
   3.1. Declaração Simples de Props 4
   3.2. Validação de Props 6
4. Emitindo Eventos (Filho → Pai) 7
5. Exercício Prático: Sistema de Feedback 9
   5.1. Fluxo de Comunicação 9
   5.2. Conceitos Aplicados 9
   5.3. Solução 10

1. Introdução

Até agora, construímos componentes que vivem de forma isolada. No entanto, o verdadeiro poder do Vue.js vem da composição desses componentes para criar aplicações complexas. Para que isso funcione, os componentes precisam conversar entre si. Neste encontro, vamos dominar o fluxo de comunicação mais fundamental do Vue: a interação entre componentes Pai e Filho.

Em Vue, dividir em componentes vs. um componente gigante muda sobretudo manutenção, reuso e performance percebida.

1.1. Por que criar componentes diferentes?

- Coesão e clareza: cada peça tem uma responsabilidade. Código menor, leitura fácil.
- Reuso real: botões, inputs, cards, tabelas → reaproveita em várias telas (menos duplicação).
- Testabilidade: unidades pequenas são fáceis de testar (unit/E2E) e de "mockar".
- Isolamento de estilos/estado: scoped CSS e estado local reduzem efeitos colaterais.
- Performance da UI: atualizações ficam localizadas; menos diffs em árvore grande.
- Code-splitting: páginas/partes podem ser carregadas sob demanda (defineAsyncComponent / rotas).
- Trabalho em equipe: menos conflitos de merge; cada dev mexe numa peça.
- Documentação e design system: facilita Storybook, catálogos de componentes, consistência visual.

1.2. Quando um único componente pode bastar?

- Widgets muito simples (pouca lógica/estado).
- Protótipos/POCs descartáveis.
- Página estática com renderização mínima.

1.3. Riscos do "componente único" (monolito)

- Arquivo enorme, difícil de entender e dar manutenção.
- Mais chance de bugs por efeitos colaterais.
- Re-renderizações abrangentes (qualquer mudança pode afetar a árvore toda).
- Dificulta testes, reuso e colaboração (muitos conflitos).

1.4. Riscos de "supercomponentizar"

- Prop drilling (props passando por 3–4 níveis). Mitigue com Pinia/composables/provide/inject.
- Muitos arquivos para algo trivial (complexidade desnecessária).
- Micro-componentes em listas enormes podem custar. Use virtualização quando renderizar milhares.

1.5. Regras práticas (rápidas)

- Se uma parte tem estado/validação própria, é reutilizável ou tem >1 responsabilidade, extraia.
- Se o SFC passar de ~200 linhas de template ou muita lógica no `<script>`, divida.
- Use `stores`/`composables` para estado compartilhado e evitar prop drilling.

Resumo: componentizar traz organização, reuso e UI mais responsiva; um único componente só é ok para casos bem pequenos e temporários.

Aqui vão exemplos rápidos (e bem práticos) de quando a comunicação entre componentes no Vue faz diferença:

a) Formulário composto: Campos-filho (`InputText`, `Select`) avisam o pai que o valor mudou/é inválido (`emit`) e o pai envia regras/valores iniciais via `props`.

b) Tabela com filtros e paginação: Barra de filtros altera um estado compartilhado (Pinia/composable); Tabela e Paginação reagem ao mesmo estado para atualizar dados.

c) Dashboard com cards: Um componente "FiltrosGlobais" muda período/segmento; vários cards e gráficos (irmãos) atualizam juntos via store ou provide/inject.

d) Wizard (passo a passo): Passos-filho emitem "próximo/anterior" e reportam validade; o container coordena qual passo mostrar e compila o payload final.

e) Modal/Drawer controlado externamente: Botões emitem "open/close" para um gerenciador de modais (store/event bus); o Modal lê o estado global para exibir/ocultar e receber conteúdo.

f) Componentes reutilizáveis com v-model: Um DatePicker custom usa `modelValue (prop)` + `update:modelValue` (`emit`) para funcionar como campo nativo, facilitando integração em qualquer form.

g) Notificações/Toasts globais: Qualquer componente dispara `notify()` (composable/store); um `ToastContainer` escuta e renderiza a fila de mensagens.

h) Layout que injeta dependências: O Layout (pai) obtém user, theme ou i18n e distribui aos descendentes com provide/inject, evitando "prop drilling".

i) Lista drag-and-drop: Itens-filho informam novas posições; o container aplica a ordenação e sincroniza com o backend.

j) Roteamento + componente de página: Um componente filho dispara eventos (ex.: "salvo") e o de página decide navegar (`router.push`), mostrar snackbar, etc.

k) Carregamento assíncrono coordenado: Header escolhe "projeto ativo"; outros componentes (Kanban, Chat, Relatórios) reagem via estado compartilhado e refazem seus fetches.

l) Slots com dados (scoped slots): Um DataFetcher busca e fornece `{ data, loading, error }` via slot; o componente que consome decide como renderizar esses estados.

2. O Fluxo de Dados Unidirecional

O Vue implementa um fluxo de dados unidirecional (one-way data flow). Isso significa que os dados têm um caminho claro e previsível:

a) Pai para Filho**:** Os dados são passados de um componente pai para um componente filho através de **Props**.

b) Filho para Pai**:** O filho não pode modificar diretamente os dados do pai. Em vez disso, ele emite Eventos para notificar o pai de que algo aconteceu, e o pai decide se deve ou não alterar seu próprio estado.

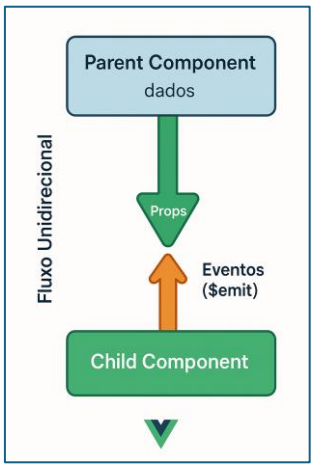


Figura 1: Diagrama do fluxo de dados unidirecional.

A figura 1 mostra como as Props fluem para baixo (Pai → Filho), enquanto eventos são emitidos para cima (Filho → Pai).

Este padrão torna o fluxo de dados da sua aplicação muito mais fácil de entender e depurar. Se um dado está incorreto, você sabe que a origem do problema está no componente pai ou no evento emitido pelo filho.

3. Passando Dados com Props (Pai → Filho)

Props (abreviação de propriedades) são atributos customizados que você pode registrar em um componente. Quando um valor é passado para um atributo de prop, ele se torna uma propriedade naquele componente filho.

3.1. Declaração Simples de Props

A forma mais simples de declarar props é usando um array de strings.

Exemplo: Componente `CartaoUsuario.vue` (Filho)

```
<template>
  <div class="cartao">
    <h3>{{ nome }}</h3>
    <p>Email: {{ email }}</p>
  </div>
</template>
```

```
<script>
export default {
  name: 'CartaoUsuario',
  // Declara as props que este componente espera receber
  props: ['nome', 'email']
}
</script>
<style scoped>
.cartao {
  border: 1px solid #ccc;
  padding: 15px;
  margin: 10px;
  border-radius: 8px;
}
</style>
```

Exemplo: Componente App.vue (Pai)

O pai pode usar o `CartaoUsuario` e passar os dados para as `props` usando `v-bind` (ou o atalho `:`).

```
<template>
  <div id="app">
    <h1>Nossa Equipe</h1>
    <CartaoUsuario
      nome="Ana Silva"
      email="ana.silva@email.com"
    />
    <CartaoUsuario
      :nome="usuarioDinamico.nome"
      :email="usuarioDinamico.email"
    />
  </div>
</template>

<script>
import CartaoUsuario from './components/CartaoUsuario.vue';

export default {
  name: 'App',
  components: { CartaoUsuario },
  data() {
    return {
      usuarioDinamico: {
        nome: 'Carlos Souza',
        email: 'carlos.souza@email.com'
      }
    }
  }
}
</script>
```

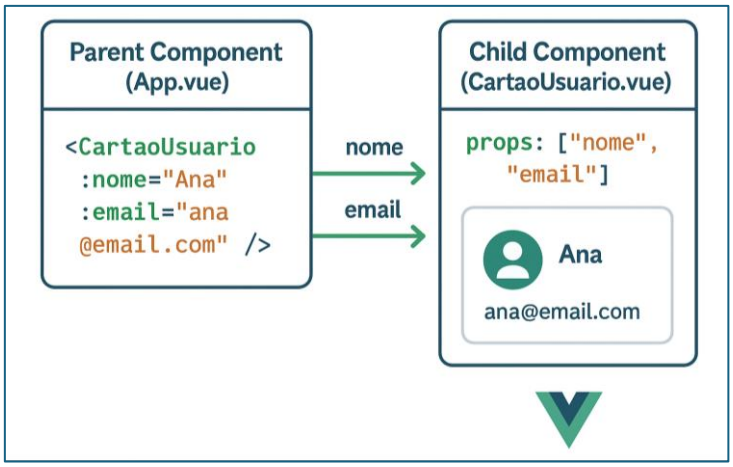


Figura 2: Comunicação de dados entre componentes pai e filho.

A figura 2 mostra o componente Pai passando dados (nome, email) para as `props` do componente Filho `CartaoUsuario`.

3.2. Validação de Props

Declarar `props` como um array de strings não oferece controle sobre o tipo de dado ou se a prop é obrigatória. Ao invés, defina suas `props` com um objeto, onde você pode especificar validações.

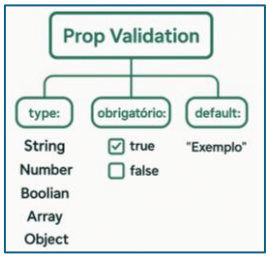


Figura 3: Validação de prop e default

A figura 3 exibe diagrama mostrando as opções de validação de uma prop: `type` (String, Number, etc.), `required` (true/false) e `default` (valor padrão).

Você pode especificar:

- `type`: O tipo de dado esperado (ex: `String`, `Number`, `Boolean`, `Array`, `Object`).
- `required`: Se a prop é obrigatória (`true` ou `false`).
- `default`: Valor padrão usado se a prop não for fornecida. Se `required` for `true`, `default` não faz sentido.
- `validator`: Uma função de validação customizada.

Exemplo: `CartaoUsuario.vue` **com validação**

```
<script>
export default {
  name: 'CartaoUsuario',
  props: {
    nome: {
      type: String,
      required: true // O nome é obrigatório
```

```
    },
    email: {
      type: String,
      required: true
    },
    idade: {
      type: Number,
      default: 18, // Se a idade não for passada, o padrão será 18
      validator: function(value) {
        // A idade deve ser maior que 0
        return value > 0;
      }
    }
  }
}
</script>
```

Se uma validação falhar (por exemplo, passar um Number para uma prop String), o Vue emitirá um aviso no console do navegador, o que é extremamente útil durante o desenvolvimento.

4. Emitindo Eventos (Filho → Pai)

Como mencionado, um filho não deve modificar diretamente uma prop. Para comunicar uma mudança ou ação de volta para o pai, o filho emite um evento customizado usando o método `this.$emit()`.

Sintaxe: `this.$emit('nome-do-evento', dadosOpcionais)`

- O primeiro argumento é o nome do evento (por convenção, usa-se o formato kebab-case).
- O segundo argumento (opcional) é um payload de dados que você quer enviar junto com o evento.

Exemplo: Componente `BotaoExcluir.vue` (Filho)

Este componente tem um botão que, ao ser clicado, notifica o pai que uma exclusão foi solicitada.

```
<template>
  <button @click="confirmarExclusao">Excluir Item</button>
</template>
<script>
export default {
  name: 'BotaoExcluir',
  methods: {
    confirmarExclusao() {
      if (confirm('Você tem certeza que deseja excluir?')) {
        this.$emit('excluir'); // Emite o evento 'excluir'
      }
    }
  }
}
</script>
```

Exemplo: Componente App.vue (Pai)

O pai pode usar o `BotaoExcluir` e ouvir (`v-on` ou `@`) o evento `excluir` para executar uma ação.

```
<template>
  <div id="app">
    <p v-if="!itemExcluido">Item para ser excluído.</p>
    <p v-else>Item foi excluído com sucesso!</p>
    <!-- Ouve o evento 'excluir' e chama o método 'handleExclusao' -->
    <BotaoExcluir @excluir="handleExclusao" />
  </div>
</template>
<script>
import BotaoExcluir from './components/BotaoExcluir.vue';
export default {
  name: 'App',
  components: { BotaoExcluir },
  data() {
    return {
      itemExcluido: false
    }
  },
  methods: {
    handleExclusao() {
      // O pai controla o que acontece quando o evento é recebido
      console.log('Evento de exclusão recebido!');
      this.itemExcluido = true;
    }
  }
}
</script>
```

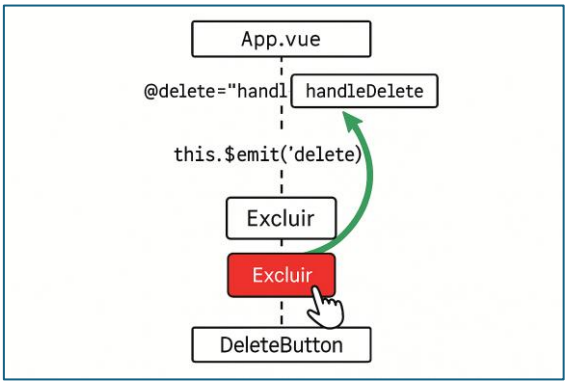


Figura 4: Ilustração do fluxo de um evento.

Na figura 4, clicar no botão do Filho dispara `this.$emit('excluir')`, que é capturado pelo `@excluir` no Pai, executando um método.

5. Exercício Prático: Sistema de Feedback

Vamos criar um sistema simples onde um componente filho (um seletor de satisfação) emite um evento para o pai, que exibe a mensagem correspondente.

a) Crie o componente `SeletorSatisfacao.vue` (Filho):

- Ele terá 3 botões: "Ruim", "Normal", "Ótimo".
- Cada botão, ao ser clicado, emitirá um evento `avaliar` com o valor correspondente (`'ruim'`, `'normal'` ou `'otimo'`).

b) Crie o componente `App.vue` (Pai):

- Ele terá uma propriedade `avaliacaoRecebida` em seu `data`.
- Ele usará o `SeletorSatisfacao` e ouvirá o evento `avaliar`.
- Quando o evento for recebido, ele chamará um método que atualiza a `avaliacaoRecebida` com o valor vindo do filho.
- Exibirá uma mensagem na tela com base na avaliação recebida (ex: "Que pena que sua experiência foi ruim." ou "Ficamos felizes que você gostou!").

Este exercício solidifica o padrão completo de comunicação Pai-Filho, a base para construir qualquer aplicação Vue complexa e bem estruturada.

5.1. Fluxo de Comunicação

O exercício demonstra o fluxo completo de comunicação entre componentes no Vue.js:

a) Usuário clica em um botão no componente `SeletorSatisfacao` (Filho).
b) Filho emite evento: `this.$emit('avaliar', nivel)` envia o nível de satisfação para o Pai.
c) Pai captura evento: `@avaliar="processarAvaliacao"` no template do `App.vue`.
d) Pai processa dados: O método `processarAvaliacao` atualiza o estado (`avaliacaoRecebida` e `mensagemFeedback`).
e) Reatividade do Vue: A interface é automaticamente atualizada, exibindo a mensagem de feedback.

5.2. Conceitos Aplicados

- Props: Embora não utilizadas neste exercício específico, você pode expandir passando um título customizado para o `SeletorSatisfacao`.
- Eventos Customizados: `$emit` para comunicação Filho → Pai.
- Renderização Condicional: `v-if` para exibir o feedback apenas após uma ação.
- Reatividade: Mudanças no data atualizam automaticamente a interface.
- Componentes Reutilizáveis: O `SeletorSatisfacao` pode ser usado em diferentes contextos.

5.3. Solução

Passo 1: Criar o Componente Filho (`SeletorSatisfacao.vue`)

Crie um novo arquivo chamado `SeletorSatisfacao.vue` dentro da pasta `src/components/` e copie o código abaixo:

```
<template>
  <div class="seletor-satisfacao">
    <h3>Como foi sua experiência?</h3>
    <div class="botoes">
      <button class="btn-ruim" @click="enviarAvaliacao('ruim')">
        😞 Ruim
      </button>
      <button class="btn-normal" @click="enviarAvaliacao('normal')">
        😳 Normal
      </button>
      <button class="btn-otimo" @click="enviarAvaliacao('otimo')">
        😊 Ótimo
      </button>
    </div>
  </div>
</template>

<script>
export default {
  name: 'SeletorSatisfacao',
  methods: {
    enviarAvaliacao(nivel) {
      // Emite o evento 'avaliar' com o nível de satisfação
      this.$emit('avaliar', nivel);
    }
  }
}
</script>

<style scoped>
.seletor-satisfacao {
  border: 2px solid #42b983;
  border-radius: 10px;
  padding: 25px;
  max-width: 500px;
  margin: 20px auto;
  background-color: #f9f9f9;
  text-align: center;
}

h3 {
  color: #2c3e50;
  margin-top: 0;
  margin-bottom: 20px;
```

```
}

.botoes {
  display: flex;
  gap: 15px;
  justify-content: center;
}

button {
  padding: 12px 24px;
  border: none;
  border-radius: 8px;
  cursor: pointer;
  font-size: 18px;
  font-weight: bold;
  transition: all 0.3s;
}

button:hover {
  transform: translateY(-2px);
  box-shadow: 0 4px 8px rgba(0,0,0,0.2);
}

.btn-ruim {
  background-color: #f5222d;
  color: white;
}

.btn-normal {
  background-color: #faad14;
  color: white;
}

.btn-otimo {
  background-color: #52c41a;
  color: white;
}
</style>
```

Explicação do Componente Filho

a) Template: Contém três botões, cada um com um emoji e texto descritivo.
b) Evento `@click`: Cada botão chama o método `enviarAvaliacao` passando o nível correspondente (`'ruim'`, `'normal'` ou `'otimo'`).
c) Método `enviarAvaliacao`: Usa `this.$emit('avaliar', nivel)` para emitir um evento customizado chamado `avaliar` e envia o nível de satisfação como payload.
d) Estilos `scoped`: Estilização isolada apenas para este componente.

Passo 2: Configurar o Componente Pai (`App.vue`)

Abra o arquivo `App.vue` e substitua todo o conteúdo pelo código abaixo:

```
<template>
  <div id="app">
    <h1>Sistema de Feedback</h1>
    <p class="subtitulo">Queremos saber sua opinião!</p>

    <!-- Componente filho que emite o evento 'avaliar' -->
    <SeletorSatisfacao @avaliar="processarAvaliacao" />

    <!-- Mensagem de feedback exibida após a avaliação -->
    <div v-if="avaliacaoRecebida" class="feedback">
      <p>{{ mensagemFeedback }}</p>
    </div>
  </div>
</template>

<script>
import SeletorSatisfacao from './components/SeletorSatisfacao.vue';
export default {
  name: 'App',
  components: {
    SeletorSatisfacao
  },
  data() {
    return {
      avaliacaoRecebida: '', // Armazena a avaliação recebida
      mensagemFeedback: ''    // Armazena a mensagem a ser exibida
    }
  },
  methods: {
    processarAvaliacao(nivel) {
      // Atualiza o estado com a avaliação recebida
      this.avaliacaoRecebida = nivel;

      // Define a mensagem de feedback baseada no nível
      if (nivel === 'ruim') {
        this.mensagemFeedback = '😔 Que pena que sua experiência foi ruim.
Vamos trabalhar para melhorar!';
      } else if (nivel === 'normal') {
        this.mensagemFeedback = '😊 Obrigado pelo feedback! Vamos continuar
melhorando nossos serviços.';
      } else if (nivel === 'otimo') {
        this.mensagemFeedback = '🎉 Ficamos felizes que você gostou! Obrigado
pela avaliação positiva!';
      }

      console.log('Avaliação recebida:', nivel);
    }
```

```
  }
}
</script>

<style>
#app {
  font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
  max-width: 700px;
  margin: 0 auto;
  padding: 40px 20px;
  text-align: center;
}
h1 {
  color: #2c3e50;
  margin-bottom: 10px;
}

.subtitulo {
  color: #666;
  font-size: 18px;
  margin-bottom: 30px;
}

.feedback {
  margin-top: 30px;
  padding: 20px;
  background: linear-gradient(135deg, #667eea 0%, #764ba2 100%);
  color: white;
  border-radius: 10px;
  box-shadow: 0 4px 15px rgba(0,0,0,0.2);
  animation: slideIn 0.5s ease-out;
}

.feedback p {
  margin: 0;
  font-size: 20px;
  font-weight: bold;
}

@keyframes slideIn {
  from {
    opacity: 0;
    transform: translateY(-20px);
  }
  to {
    opacity: 1;
    transform: translateY(0);
  }
}
</style>
```

Explicação do Componente Pai

- Importação: Importa o componente `SeletorSatisfacao`.
- Registro: Registra o componente na propriedade `components`.
- Uso do Componente: `<SeletorSatisfacao @avaliar="processarAvaliacao" />` - Ouve o evento `avaliar` emitido pelo filho.
- `Data`:
  - `avaliacaoRecebida`: Armazena qual avaliação foi feita (vazio inicialmente).
  - `mensagemFeedback`: Armazena a mensagem a ser exibida.
- Método `processarAvaliacao`: Recebe o nível de satisfação e define a mensagem apropriada.
- Renderização Condicional: `v-if="avaliacaoRecebida"` garante que o feedback só apareça após uma avaliação ser feita.